

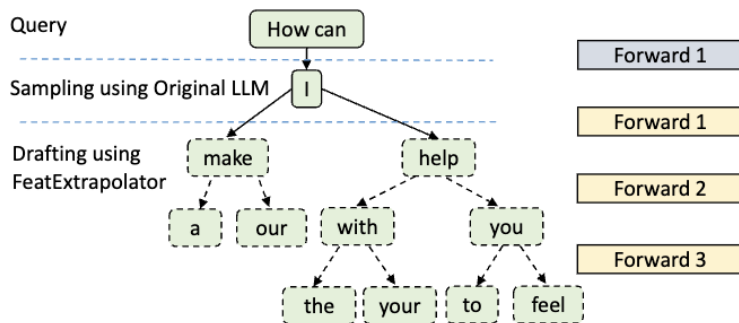
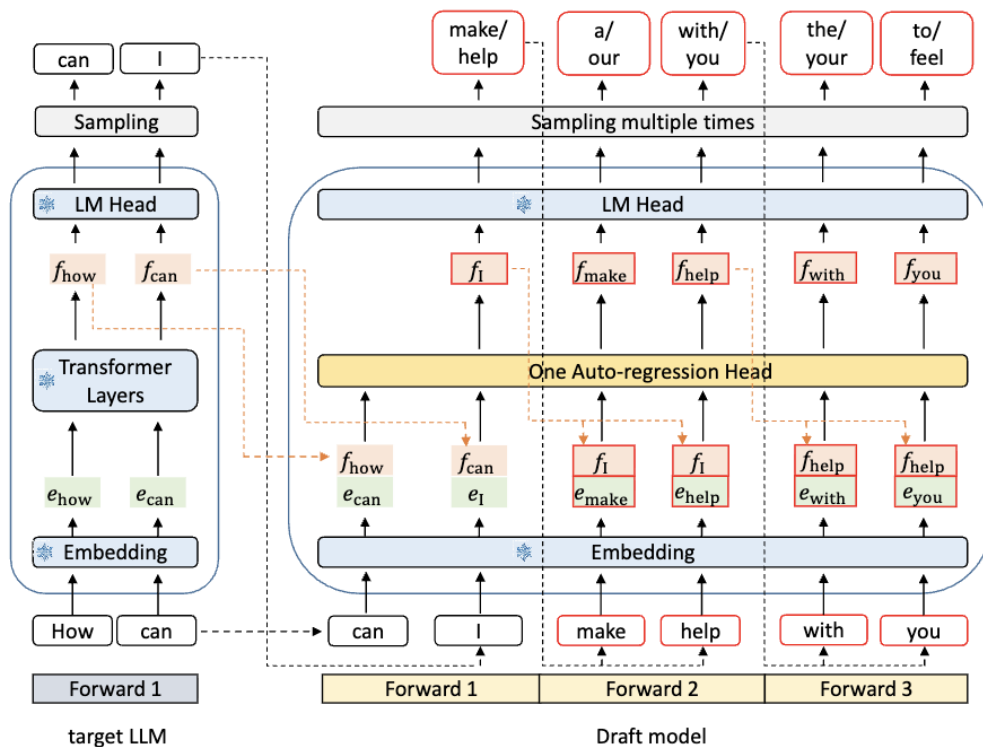


University
of Exeter

COMM117

Large Language Models and Application

Paper Discussion: Evaluation



		HumanEval		GSM8K		Alpaca	
	Model	Speedup	τ	Speedup	τ	Speedup	τ
T=0	V 7B	3.33x	4.29	3.01x	4.00	2.79x	3.86
	V13B	3.58x	4.39	3.08x	3.97	3.03x	3.95
	V 33B	3.67x	4.28	3.25x	3.94	2.97x	3.61
	LC 7B	3.17x	4.24	2.91x	3.82	2.78x	3.71
	LC 13B	3.76x	4.52	3.20x	4.03	3.01x	3.83
	LC 70B	3.52x	4.42	3.03x	3.93	2.97x	3.77
T=1	V 7B	2.39x	3.43	2.34x	3.29	2.21x	3.30
	V13B	2.65x	3.63	2.57x	3.60	2.45x	3.57
	V 33B	2.76x	3.62	2.77x	3.60	2.52x	3.32
	LC 7B	2.61x	3.79	2.40x	3.52	2.29x	3.33
	LC 13B	2.89x	3.78	2.82x	3.67	2.66x	3.55
	LC 70B	2.92x	3.76	2.74x	3.58	2.65x	3.47

Table 2: Average acceptance length τ and acceptance rate α on MT-bench. T denotes temperature.

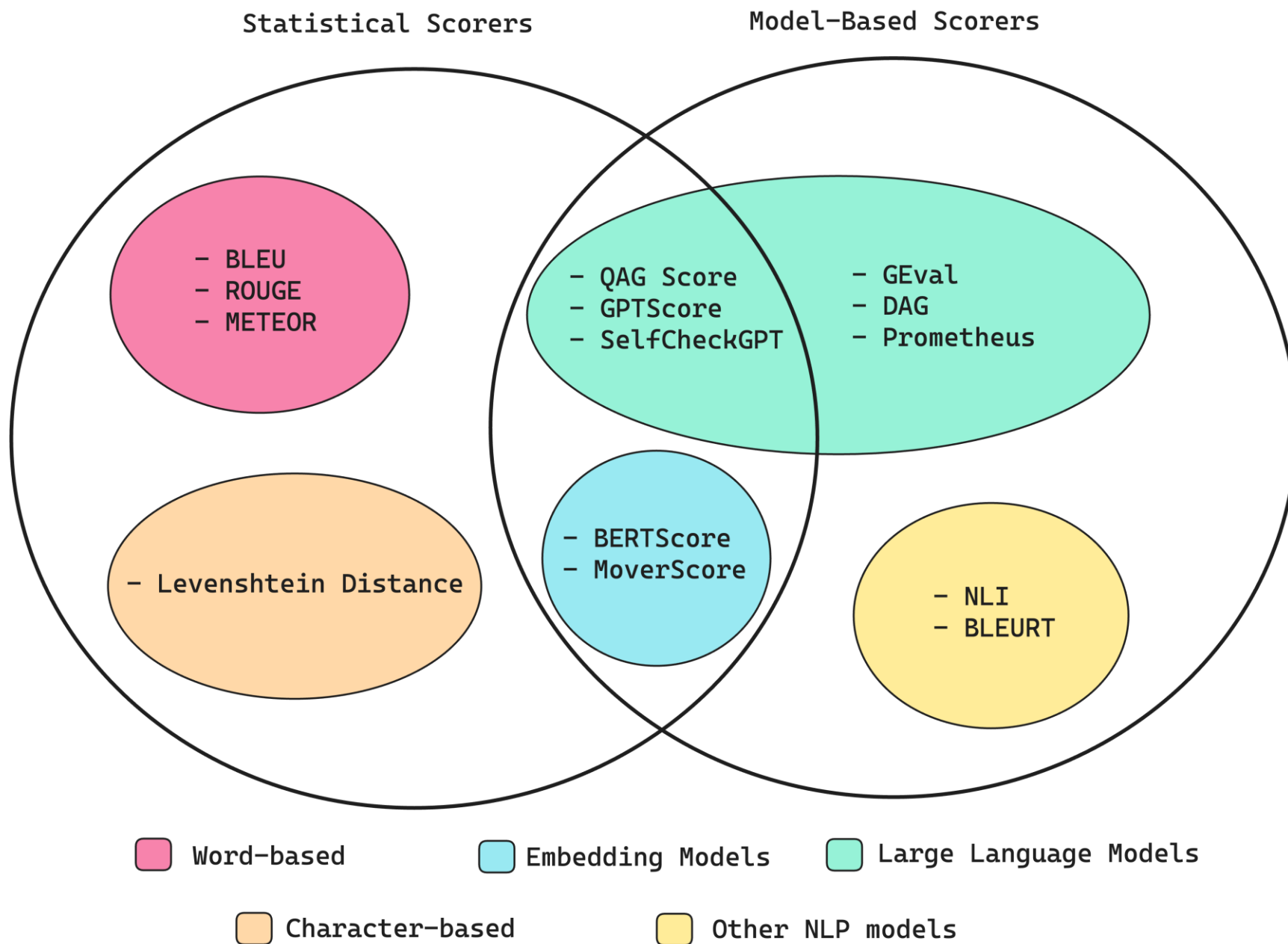
		Model	τ	0- α	1- α	2- α	3- α	4- α
T=0		Vicuna 7B	3.94	0.79	0.74	0.72	0.73	0.67
		Vicuna 13B	3.98	0.79	0.74	0.72	0.74	0.70
		Vicuna 33B	3.68	0.74	0.69	0.67	0.67	0.66
		LLaMA2-Chat 7B	3.62	0.76	0.69	0.67	0.68	0.68
		LLaMA2-Chat 13B	3.90	0.77	0.69	0.69	0.70	0.71
		LLaMA2-Chat 70B	3.81	0.75	0.69	0.65	0.64	0.64
T=1		Vicuna 7B	3.17	0.71	0.68	0.66	0.66	0.65
		Vicuna 13B	3.20	0.73	0.68	0.68	0.67	0.69
		Vicuna 33B	3.22	0.71	0.67	0.64	0.64	0.64
		LLaMA2-Chat 7B	3.30	0.71	0.66	0.66	0.66	0.64
		LLaMA2-Chat 13B	3.45	0.73	0.69	0.66	0.67	0.67
		LLaMA2-Chat 70B	3.46	0.73	0.67	0.64	0.66	0.65

Eva

- “Bet
- Diffe
- Goc

Ho

- Ben
- Hun
- Syst



SS

Evaluation: Semantic evaluation

- Embedding cosine similarity
- BERT Score

Cosine Similarity

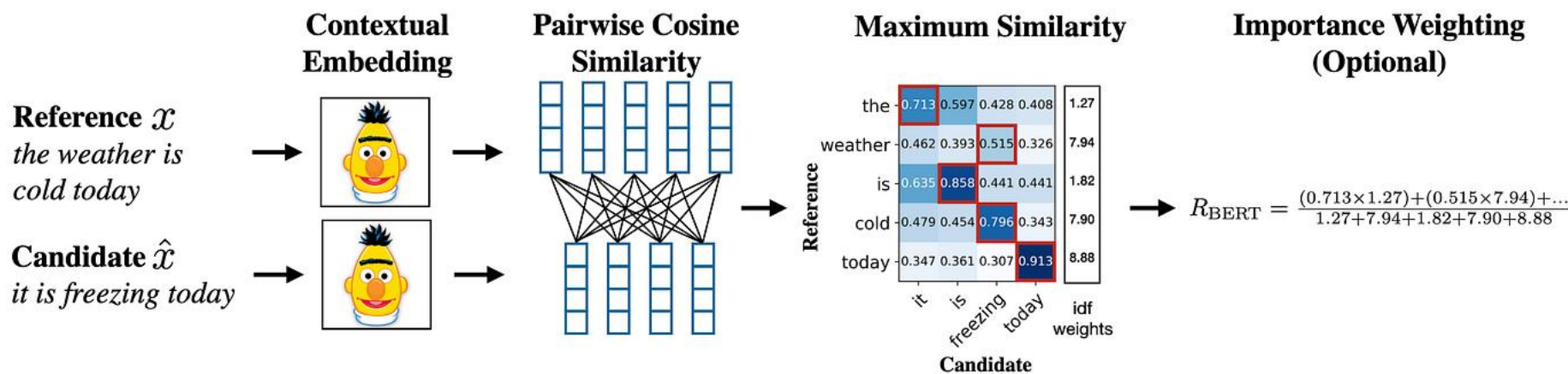
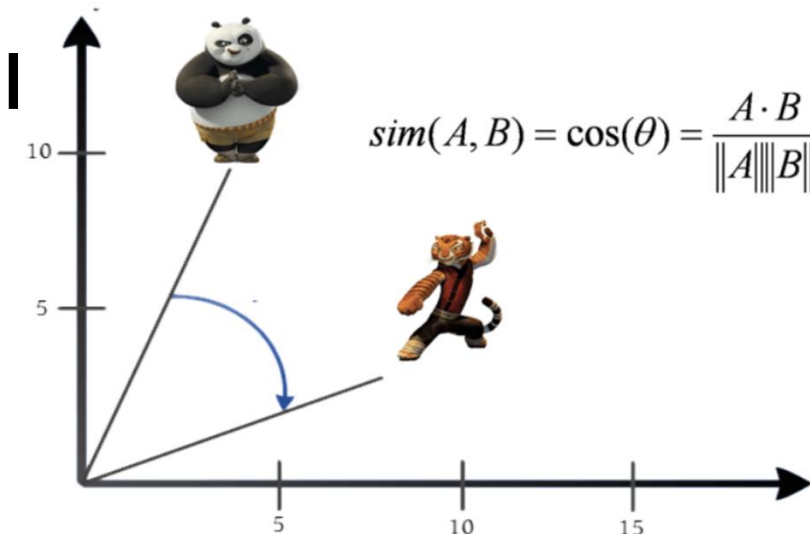


Figure 1: Illustration of the computation of the recall metric R_{BERT} . Given the reference x and candidate $\hat{x}$, we compute BERT embeddings and pairwise cosine similarity. We highlight the greedy matching in red, and include the optional idf importance weighting.

Evaluation: System Level

Latency

- TTFT (time to first token)
- Total latency
- p50 / p95 / p99

Throughput / cost

- tokens/sec (output tokens/sec)
- requests/sec (RPS)
- GPU memory usage / concurrency
- \$/1k tokens (if hosted)

Evaluation: System Level

Reproducibility checklist

Must report:

- hardware (GPU type), precision/quantisation
- batching/concurrency strategy
- prompt/output length distribution
- decoding settings
- warm vs cold runs

Evaluation: System Level

Why system-level metrics are non-negotiable

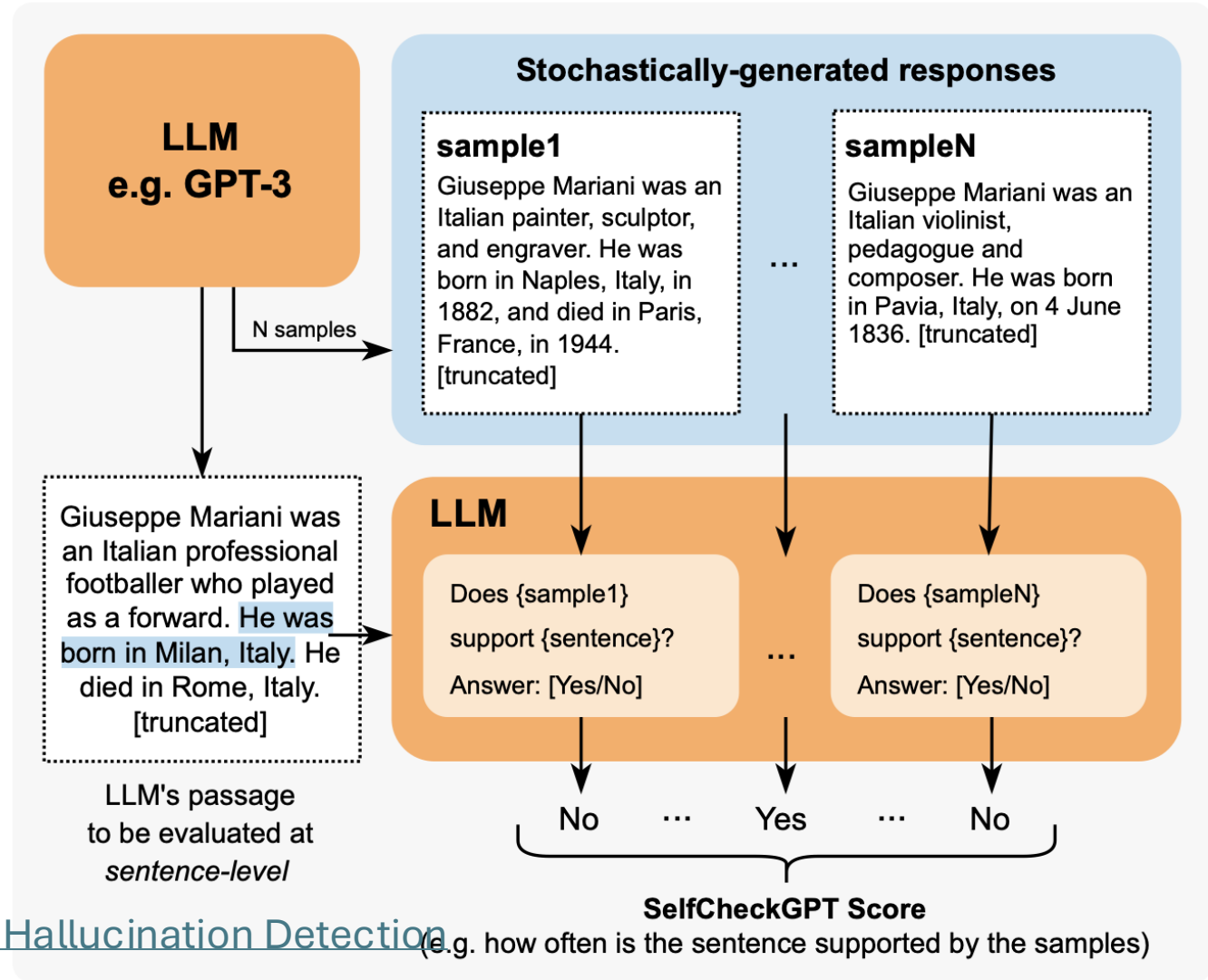
Same algorithm can look better/worse depending on:

- batching strategy (static/dynamic/continuous)
- prompt length mix
- temperature / decoding method
- hardware memory bandwidth

Evaluation: QAG Score

QAG (Question Answering)
LLMs' high reasoning

- Use an LLM to extract
- For each claim, ask to support or not ('no') with the claim



[SELFCKGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models](#)

Evaluation:

- Use **automatic** metrics for correctness when ground truth exists
- Use **human** eval for preference/safety/open-ended quality
- Use **system** eval for real-world performance
- Always match metric $\leftrightarrow$ question $\leftrightarrow$ workload

Introduction Today

- 1.Extract summarisation
- 2.Abstractive summarisation

What is summarization

- Compress a document while preserving **key information**
- Constraints: length, audience, style, coverage

It requires the ability:

1. Understanding the context and semantic relationships
2. Generating a shorter coherent version that capture the core idea of the document.

“Text summarization is a powerful tool that can speed up various business processes by relieving the burden of domain experts to read long documents in detail.”

What “good” looks like

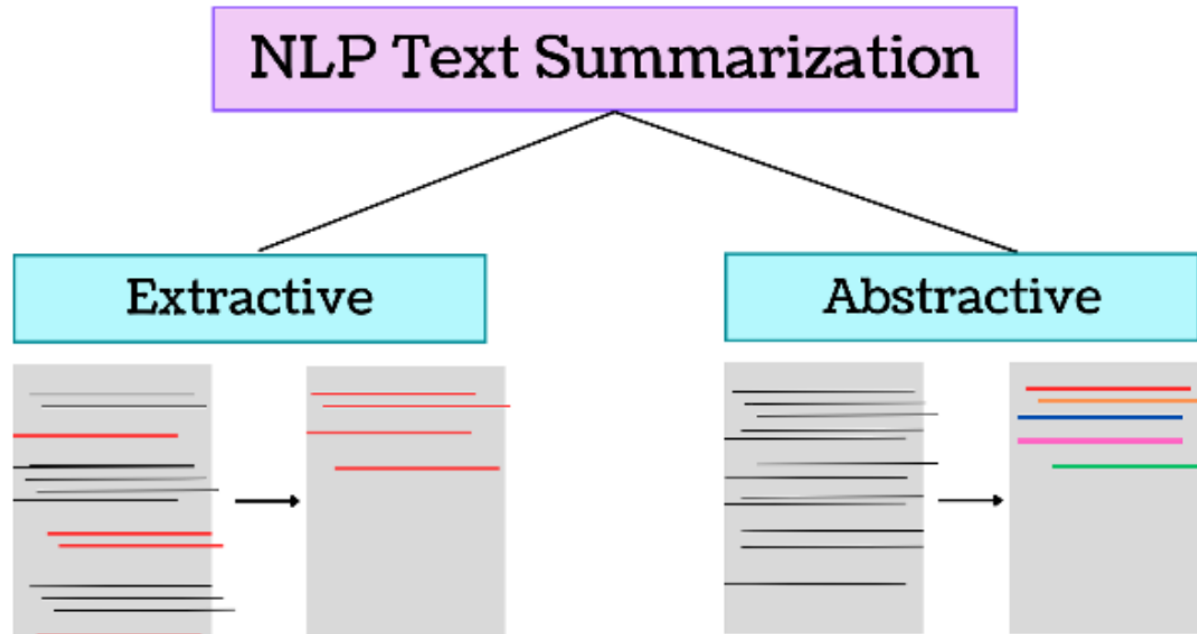
- ✓ **Coverage:** includes key points
- ✓ **Faithfulness:** no invented facts
- ✓ **Clarity:** coherent and readable

Common failure models

- Missing key point (low coverage)
- Redundancy (repeats)
- Hallucination (adds facts)
- Distortion (subtle incorrect details)

Two main types

- **Extractive**: select sentences/spans from source
- **Abstractive**: generate new wording (paraphrase + fusion)



Extractive

It simply **takes out the important sentences or phrases** from the **original text** and joins them to form a summary.

→ Output is a subset of original sentences

Great when:

Correctness is critical, legal/medical notes



Extractive

It simply **takes out the important sentences** or phrases from the **original text** and joins them to form a summary.

How to choose the important sentences?

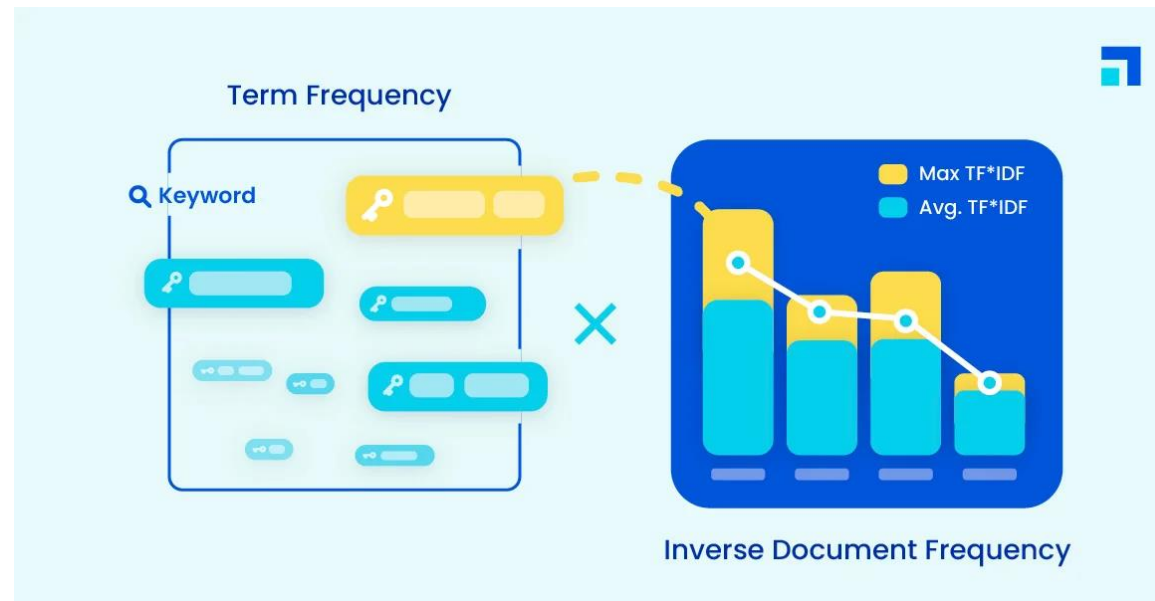
Ranking algorithm!

Assigns scores to each sentence in the text based on their relevance to the overall meaning of the document



TF - IDF Term Frequency-Inverse Document Frequency

“ It is a **numerical statistic** that is intended to reflect how important a word is to a document in a collection or corpus.”



Term Frequency

It is a measure of how frequently a term, t , appears in a document, d :

$$tf_{t,d} = \frac{n_{t,d}}{\text{Number of terms in the document}}$$

TF - IDF Term Frequency-Inverse Document Frequency

Term Frequency

It is a measure of how frequently a term, t , appears in a document, d :

$$tf_{t,d} = \frac{n_{t,d}}{\text{Number of terms in the document}}$$

Example: "This movie is not scary and is slow"

Vocabulary: 'This', 'movie', 'is', 'very', 'scary', 'and', 'long', 'not', 'slow', 'spooky', 'good'

1. 1. 2. 0. 1. 1. 0. 1. 1. 0. 0

TF:	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{2}{8}$	0	$\frac{1}{8}$	$\frac{1}{8}$	0	$\frac{1}{8}$	$\frac{1}{8}$	0	0
-----	---------------	---------------	---------------	---	---------------	---------------	---	---------------	---------------	---	---

TF - IDF Term Frequency-Inverse Document Frequency

Inverse Document Frequency (IDF)

IDF is a measure of how important a term is.

$$idf_t = \log \frac{\text{number of documents}}{\text{number of documents with term 't'}}$$

Example:

Review 1: This movie is very scary and long

Review 2: This movie is not scary and is slow

Review 3: This movie is spooky and good

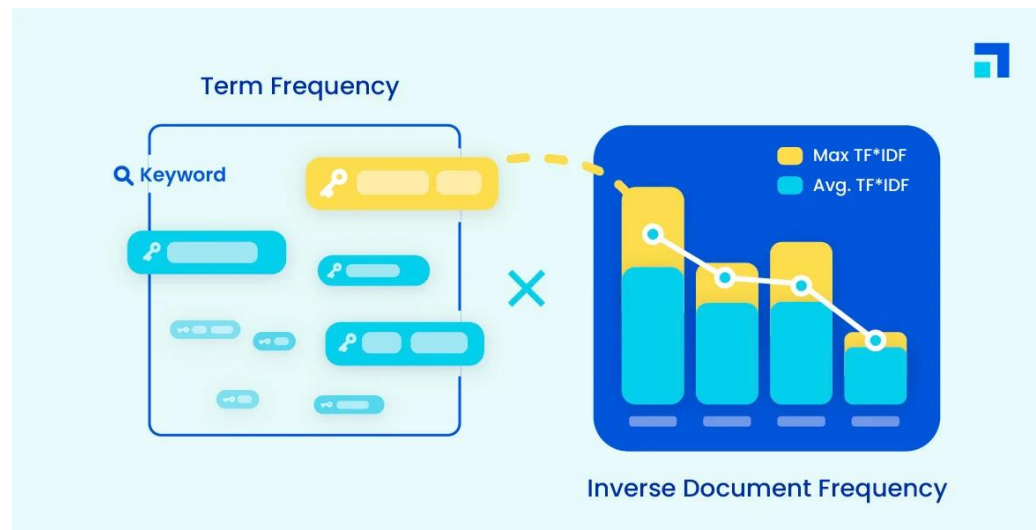
For Review 2: 'This', 'movie', 'is', 'not', 'scary', 'and', 'is', 'slow',
3. 3. 3. 1. 2. 3. 1

IDF: $\log \frac{3}{3}$ $\log \frac{3}{3}$ $\log \frac{3}{3}$ $\log \frac{3}{1}$ $\log \frac{3}{2}$ $\log \frac{3}{3}$ $\log \frac{3}{1}$

IDF ↑

Term	Review 1	Review 2	Review 3	IDF
This	1	1	1	0.00
movie	1	1	1	0.00
is	1	2	1	0.00
very	1	0	0	0.48
scary	1	1	0	0.18
and	1	1	1	0.00
long	1	0	0	0.48
not	0	1	0	0.48
slow	0	1	0	0.48
spooky	0	0	1	0.48
good	0	0	1	0.48

TF - IDF Term Frequency-Inverse Document Frequency



$$(tf_idf)_{t,d} = tf_{t,d} * idf_t$$

For Review 2: 'This', 'movie', 'is', 'not', 'scary', 'and', 'slow',

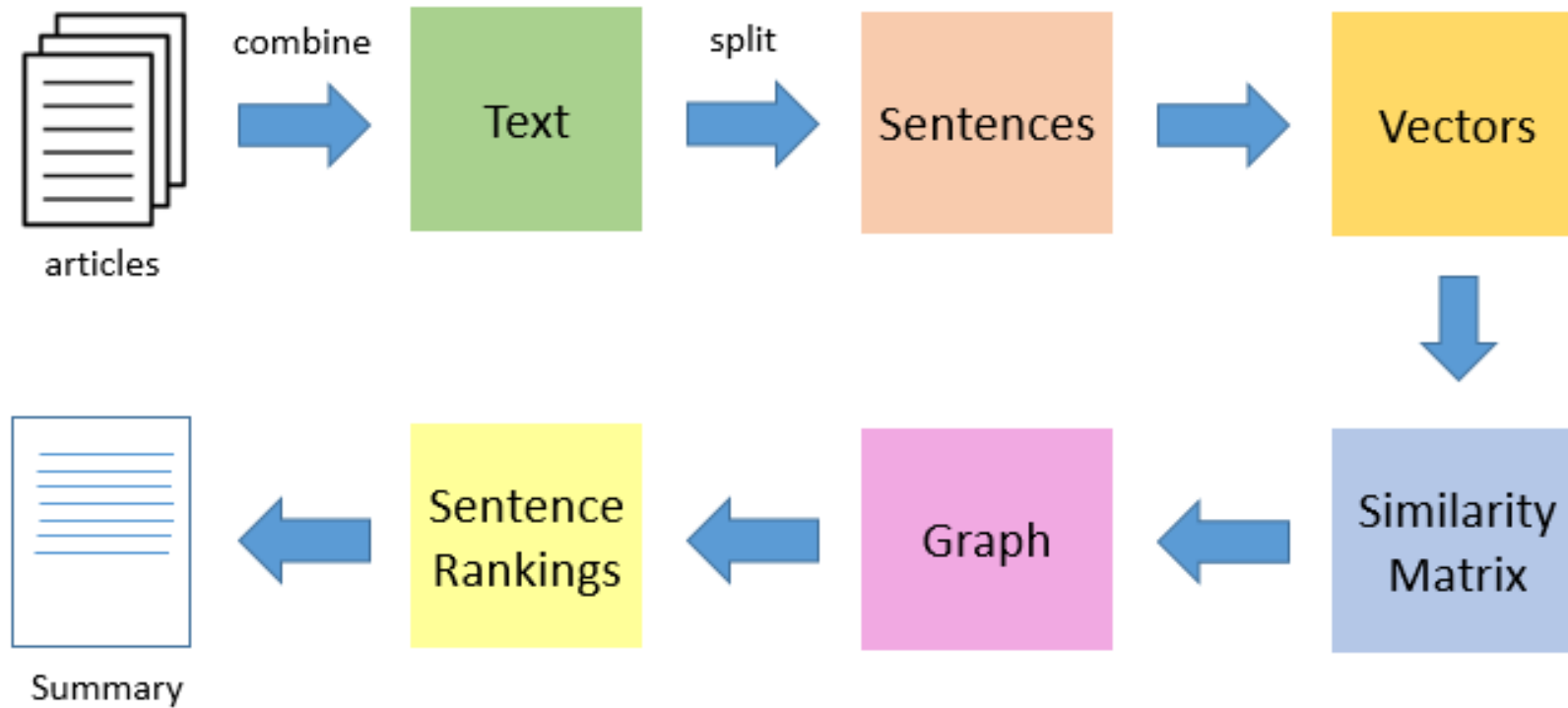
TF:	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{2}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$
*							
IDF:	$\log \frac{3}{3}$	$\log \frac{3}{3}$	$\log \frac{3}{3}$	$\log \frac{3}{1}$	$\log \frac{3}{2}$	$\log \frac{3}{3}$	$\log \frac{3}{1}$

→ 0 0 0 0.06 0.023 0 0

TF - IDF Term Frequency-Inverse Document Frequency

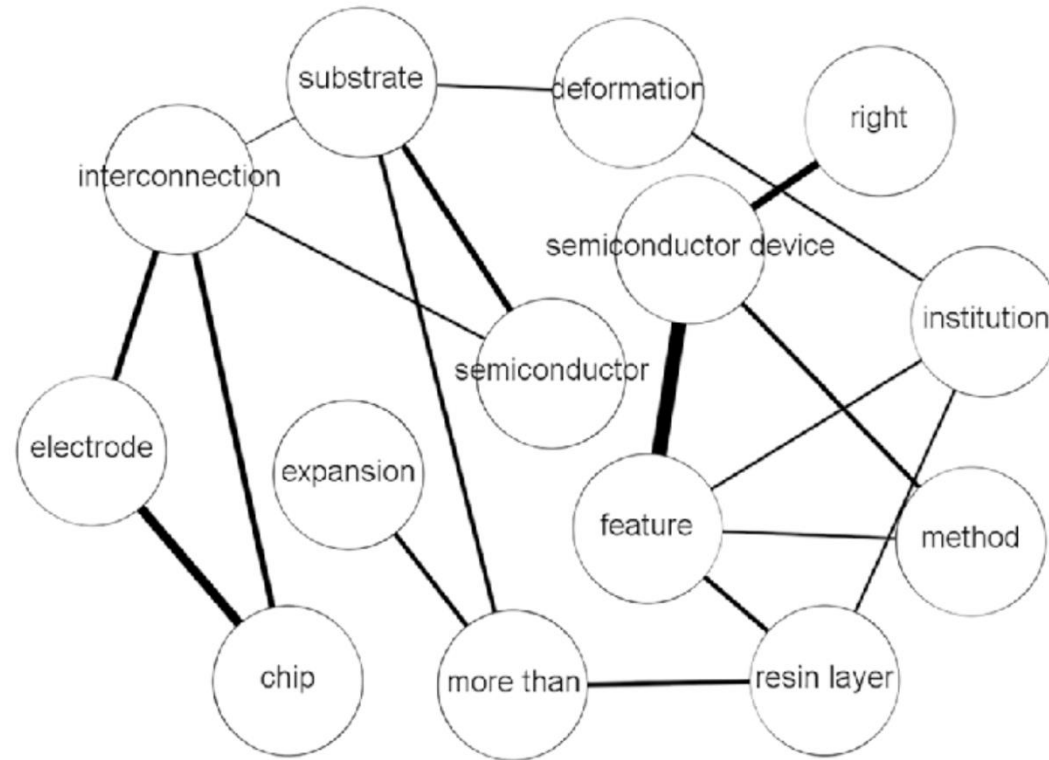
```
# importing TfidfVectorizer class to convert a collection of raw documents to a matrix of TF-IDF features.
from sklearn.feature_extraction.text import TfidfVectorizer
# importing cosine_similarity function to compute the cosine similarity between two vectors.
from sklearn.metrics.pairwise import cosine_similarity
from heapq import nlargest
def generate_summary(text, n):
    sentences = sent_tokenize(text)
    # Create the TF-IDF matrix
    vectorizer = TfidfVectorizer(stop_words='english')
    tfidf_matrix = vectorizer.fit_transform(sentences)
    # Compute the cosine similarity between each sentence and the document
    sentence_scores = cosine_similarity(tfidf_matrix[-1], tfidf_matrix[:-1])[0]
    # Select the top n sentences with the highest scores
    summary_sentences = nlargest(n, range(len(sentence_scores)), key=sentence_scores.__getitem__)
    summary_tfidf = ' '.join([sentences[i] for i in sorted(summary_sentences)])
    return summary_tfidf
```

TextRank (graph-based)



TextRank (graph-based)

- Sentences as nodes
- Edge weight = similarity
- **PageRank** selects central sentences
- Often reduces redundancy vs pure TF-IDF



TextRank (graph-based)

PageRank

- Suppose we have 4 web pages.
- To capture the probabilities of users navigating from one page to another, create the matrix M
- Each element of this matrix denotes the probability of a user transitioning from one web page to another.

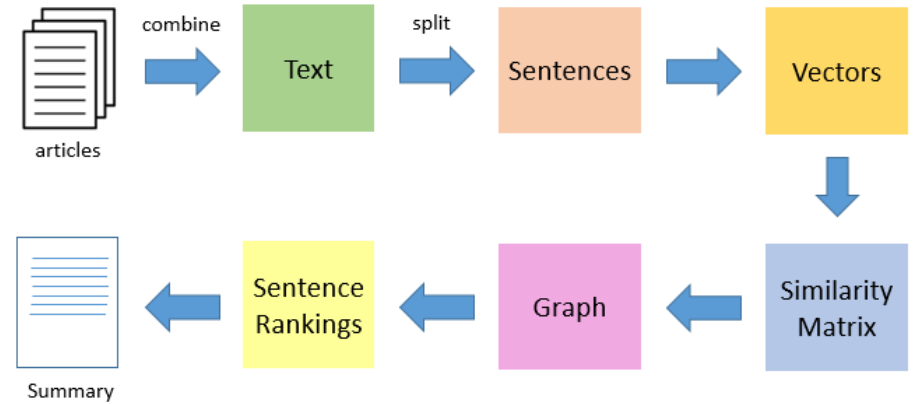
M =

	w1	w2	w3	w4
w1				
w2				
w3				
w4				

P(w1 to w2)

M =

	w1	w2	w3	w4
w1	0	0.5	0	0.5
w2	0.5	0	0.5	0
w3	0.25	0.25	0.25	0.25
w4	1	0	0	0



TextRank (graph-based)

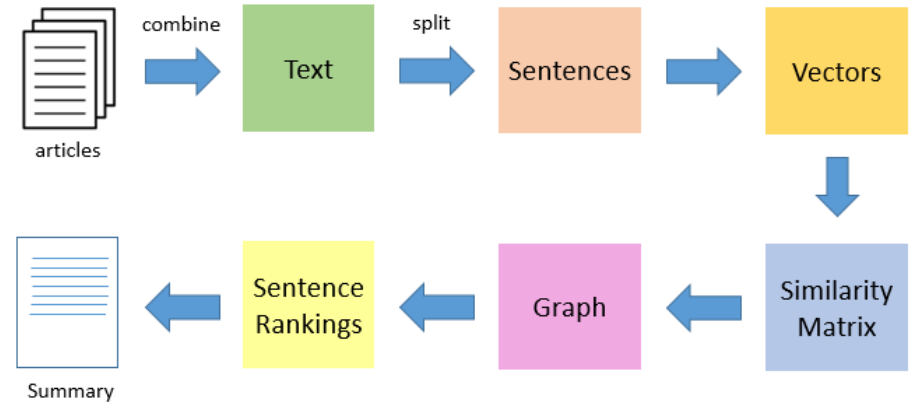
PageRank

- Suppose we have 4 web pages.
- To capture the probabilities of users navigating from one page to another, create the matrix M
- Each element of this matrix denotes the probability of a user transitioning from one web page to another.



TextRank

- In place of web pages, we use sentences
- **Similarity between any two sentences** is used as an equivalent to the web page transition probability
- The **similarity scores** are stored in a square matrix, similar to the matrix M used for PageRank



TextRank (graph-based)

PageRank

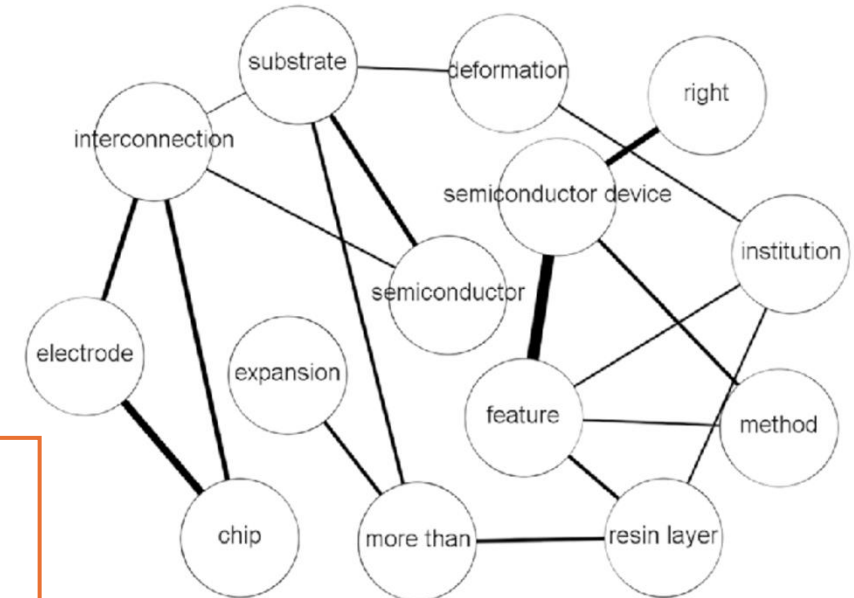
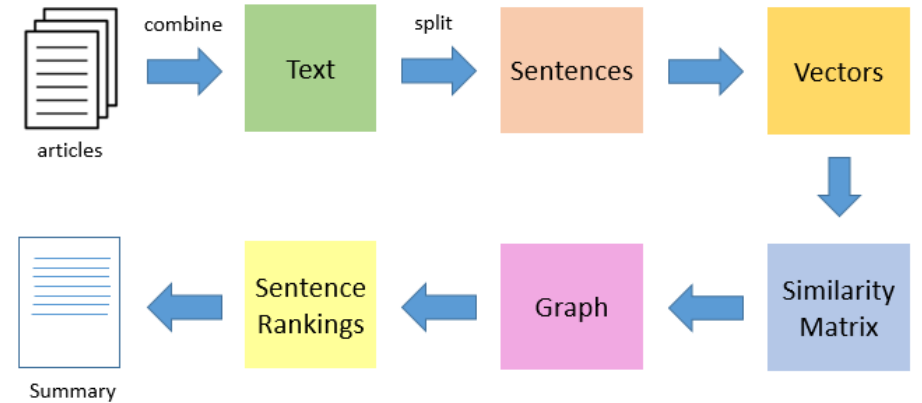
- Suppose we have 4 web pages.
- To capture the probabilities of users navigating from one page to another, create the matrix M
- Each element of this matrix denotes the probability of a user transitioning from one web page to another.



TextRank

- In place of web pages, we use sentences
- **Similarity between any two sentences** is used as an equivalent to the web page transition probability
- The **similarity scores** are stored in a square matrix, similar to the matrix M used for PageRank

If the sentence is similar to many key sentences, it receives many weighted votes and gets high rank → likely selected into summary.



TextRank (graph-based)

Extract word vectors

```
word_embeddings = {}
```

```
f = open('glove.6B.100d.txt', encoding='utf-8')
```

```
for line in f:
```

```
    values = line.split()
```

```
    word = values[0]
```

```
    coefs = np.asarray(values[1:], dtype='float32')
```

```
    word_embeddings[word] = coefs
```

```
f.close()
```

#Similarity Matrix Preparation

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
for i in range(len(sentences)):
```

```
    for j in range(len(sentences)):
```

```
        if i != j:
```

```
            sim_mat[i][j] = cosine_similarity(sentence_vectors[i].reshape
```

```
(1,100), sentence_vectors[j].reshape(1,100)) [0,0]
```

Apply PageRank

```
import networkx as nx
```

```
nx_graph = nx.from_numpy_array(sim_mat)
```

```
scores = nx.pagerank(nx_graph)
```



US Open tennis player Serena Williams in action against Coco Gauff. Williams is the first of the Big Four to play at the US Open since 2008. She defeated Gauff 6-3, 6-2 to win the 2023 US Open women's singles title. (Photo: Getty Images)

RELEASED: OCT 20, 2023, 10:10 PM EDT | UPDATED: OCT 20, 2023, 10:41 AM
in Chat

GARFUNKEL - Miss Williams of Tennessee was the biggest tale of two coasts on Sunday (Oct. 15) defeating Gauff 6-3, 6-2, 6-2 to win the 2023 US Open women's singles title. Williams, 35, is the first of the Big Four to play at the US Open since 2008. She defeated Gauff 6-3, 6-2 to win the 2023 US Open women's singles title. (Photo: Getty Images)

She is the first of the Big Four to play at the US Open since 2008. She defeated Gauff 6-3, 6-2 to win the 2023 US Open women's singles title. (Photo: Getty Images)



Rafael Nadal and Roger Federer pose during retirement. (Image: Getty Images)

RELATED ARTICLES

- Paris Masters results LIVE: Federer, Djokovic and Djokovic still in action
- Maria Cilic makes Federer, Djokovic and Djokovic predictions

"So tennis needs a new generation to win Grand Slams while the Big Four are still playing."

"Otherwise, (Alexander) Zverev and Cilic won't have credibility."



ASHEVILLE, NC (WSPA) - Fed Cup tennis is returning to Asheville for the second consecutive year according to the USTA.

The US Cellular Center in Asheville will host the United States' Fed Cup First Round tie against Australia in February 2018.

The matches will take place on February 9 and 10.

"We couldn't be happier to be bringing Fed Cup back to Asheville," said USTA President Katrina Adams. "Everything about the First Round tie vs. the Netherlands corner this year was incredible. I see local business partners to the staff at the U.S. Cellular Center to the local volunteers and the entire tennis community that cheered us on to victory. Asheville made for a perfect Fed Cup Host. I was looking forward to coming back."

But the裴森 crowd appeared to not know what had happened at first and started booing before delivering a slow-hand clap.

They then applauded as the chair umpire returned to his seat.

The man was eventually walked away from his seat by medical staff.



Novak Djokovic was composed as he moved through the Paris Masters second round. (Image: Getty Images)

Djokovic ran out a 7-5 6-1 winner over Sousa to progress to the third round in the French capital.

He will play world No. 52 Denis Shapovalov, with potential quarter-finals clash with Maria Cilic in the horizon.

Novak Djokovic is hoping to win the Paris Masters for the fifth time in his career.

<https://www.analyticsvidhya.com/blog/2018/11/introduction-text-summarization-text>

Strengths and limitations

Strengths

- High faithfulness
- Simple + cheap
- Easier evaluation with ROUGE

Limitations

- Can be **choppy**
- **Hard to merge** across sentences
- Can include **irrelevant context around** selected sentences

Abstractive summarisation

- Model generates a **new summary** from the source
- Can **paraphrase** and **merge** information

Better readability, but riskier

ChatGPT primarily does *abstractive* summarisation.

Abstractive summarisation

- Model generates a **new summary** from the source
- Can **paraphrase** and **merge** information

Better readability, but riskier

ChatGPT primarily does *abstractive* summarisation ?

- **Prompt for Extractive:**

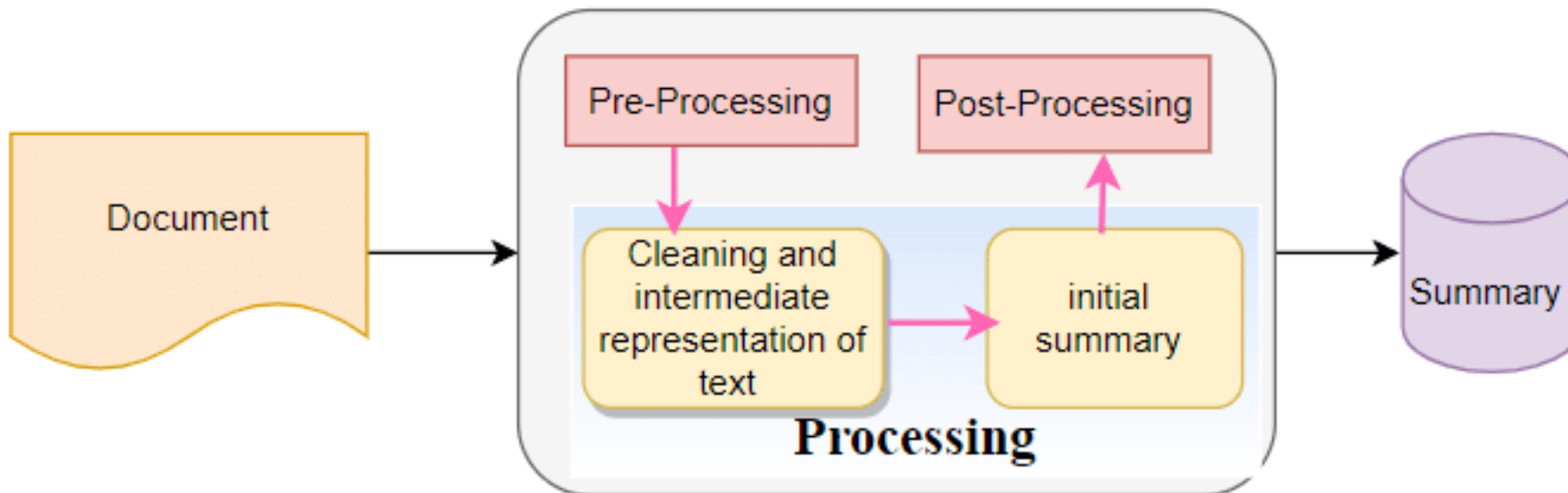
Highlight the most important sentences from the text without rewriting.

- **Prompt for Abstractive:**

Summarize the text in 3 sentences in your own words.

Abstractive summarisation

- Encoder-decoder (seq2seq) or instruction-tuned LLM
- “Read” source → generate summary tokens
- Controlled by prompts and decoding (temperature, top-p)



Abstractive summarisation

Why Abstractive summarisation is better?

Extractive summarization is more rigid and might read awkward.

- Text generation in abstractive summarization **imitates better human writing style** and is **easier to read**.
- Abstractive summary is **more coherent and concise**.
- The text generation used in the summarization model gives us **more flexibility** in the future **to apply it to other cases**.



Abstractive summarisation

However...

- **Hallucination**

The summary includes information that is **not supported by the source text**.

- **Over-generalization**

The summary makes a claim **broader/stronger** than what the source actually says.

- **Wrong numbers/names/dates** (high-stakes issue)

The summary changes **factual details** like:
percentages, counts, dates, people/organization names, locations.

Abstractive summarisation

Prompting template for summarisation (practical)

- System: “faithful summariser”
- User: include source + audience + length
- Constraints: “no new facts; cite sentences if possible”
- Output: bullet summary / JSON fields

Evaluation summarisation

Why evaluation is hard for summarisation

- **Many valid summaries**

Unlike classification (one correct label), summarisation usually has **multiple correct answers**.

→ If you compare only to **one reference summary**, a **good summary** may **score low** just because it chose different wording or focus.

Evaluation summarisation

Why evaluation is hard for summarisation

- Many valid summaries
- **Lexical overlap $\neq$ meaning**

Metrics like ROUGE/BLEU mostly reward **word overlap** with the reference.

- Reference:

“The model reduces memory fragmentation during inference.”

- Candidate summary:

“The approach lowers memory waste while generating outputs.”

Similar meaning but little exact overlap

Evaluation summarisation

Why evaluation is hard for summarisation

- Many valid summaries
- Lexical overlap $\neq$ meaning
- **Faithfulness matters more than style**

A summary can be: very fluent, nicely structured, concise, and still be **factually wrong**. → **faithfulness** (is it supported by source?) is often **more important than style**.

- Source :
“The method improved **throughput** by 2–4× on tested **workloads**.”
- Bad summary: 
“The method consistently improves **accuracy** by 2–4× across **tasks**.”

Evaluation summarisation

Why evaluation is hard for summarisation

- Many valid summaries
- Lexical overlap $\neq$ meaning
- **Faithfulness matters more than style**

A summary can be: very fluent, nicely structured, concise, and still be **factually wrong**. → **faithfulness** (is it supported by source?) is often **more important than style**.

- Source :
“The method improved **throughput** by 2–4× on tested **workloads**.”
- Bad summary: 
“The method consistently improves **accuracy** by 2–4× across **tasks**.”

Automatic metrics: ROUGE (quick)

- ROUGE-1/2/L = n-gram overlap
- Works better for extractive

ROUGE is weaker for:

paraphrases

abstractive summaries

factual consistency checks

Semantic metrics help

Embedding/BERTScore-style metrics can capture meaning similarity better

But:

- they may still reward fluent hallucinations
- they don't truly verify source grounding

Human / verifier-based methods matter

That's why summarisation evaluation often combines:

- ✓ **ROUGE** (content overlap)
- ✓ **semantic similarity**
- ✓ **faithfulness checks** (QA/NLI/LLM-judge)

QA based check

“does summary answer same questions as source?”

- ✓ **human rubric** (coverage, faithfulness, clarity)

Take Away

- Extractive: select sentences → faithful, less fluent
- Abstractive: generate paraphrase → fluent, hallucination risk
- Evaluate with ROUGE + semantic metrics + human rubric when needed

Next steps

1. LLM-based translation techniques; Prompting vs fine-tuned models
2. RAG

Mid term feedback



Additional Material:

Core summarisation evaluation papers

- [ROUGE: A Package for Automatic Evaluation of Summaries \(Lin, 2004\)](#)
Classic overlap-based summarisation metric (ROUGE-1/2/L). Great for understanding the baseline everyone reports.
- [Asking and Answering Questions to Evaluate the Factual Consistency of Summaries](#) (Wang, Cho, Lewis, ACL 2020)
QA-based factual consistency evaluation for summaries (good bridge to your “faithfulness” discussion).
- [QuestEval: Summarization Asks for Fact-based Evaluation](#) (Scialom et al., EMNLP 2021)
QA/QG-based fact-oriented evaluation; useful as a “beyond ROUGE” paper.

Semantic metric paper

- [BERTScore](#) (Zhang et al., ICLR 2020)
Meaning-aware automatic metric using contextual embeddings (good to contrast with ROUGE).

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.



r.mu2@exeter.ac.uk

Questions?